



CURSO

Bases de Datos II

Proyecto #2

Profesor

Kenneth Roberto Obando Rodriguez

Estudiantes

Danielo Wu Zhong | 2023150448

Beatriz Rebeca Díaz Gómez | 2024090972

I CICLO, 2026

Tabla de Contenidos

Descripción General del Proyecto.....	4
Componentes Principales.....	4
Tecnologías del Proyecto 1 (fuentes de datos).....	4
Arquitectura del Sistema.....	5
Diagrama de Arquitectura.....	5
Infraestructura Docker.....	6
Contenedores.....	6
Redes Docker.....	6
Volúmenes Persistentes.....	7
Cadena de Dependencias de Arranque.....	7
Dockerfiles Personalizados.....	7
Interfaces de Acceso.....	8
Data Warehouse en Apache Hive.....	9
Esquema Estrella.....	9
Dimensiones.....	10
Tablas de Hechos.....	12
Estrategia de Llaves Surrogate (CRC32).....	13
Vistas OLAP.....	14
Vista 1: olap_ingresos_por_mes_categoria.....	14
Vista 2: olap_actividad_usuarios_por_restaurante.....	14
Vista 3: olap_estado_pedidos.....	15
Vista 4: olap_tendencias_consumo.....	15
Vista 5: olap_horarios_pico.....	16
Vista 6: olap_crecimiento_mensual.....	16
Pipeline ETL con Apache Airflow.....	17
Flujo de Tareas.....	17
Descripción de cada Tarea.....	18
Configuración Compartida de Spark (SPARK_CONF).....	19
Conexiones Requeridas en Airflow.....	19
Procesamiento con Apache Spark.....	20
Archivos relacionados.....	20
Flujo de procesamiento.....	20
Uso de DataFrames y SparkSQL.....	21
Resultados analíticos generados.....	22
Parámetros del job restaurants_spark_analytics.py.....	23
Comando de validación en Windows CMD.....	24
Análisis de Grafos con Neo4J.....	25
Archivos relacionados.....	25
Estructura del grafo.....	25

Carga del grafo.....	27
Consultas de análisis implementadas.....	27
Comandos de validación en Windows CMD.....	28
Consultas visuales en Neo4J Browser.....	29
Asignación de Rutas de Entrega.....	31
Heurística utilizada.....	31
Validación visual de rutas.....	31
Visualización con Metabase.....	33
Dashboards Disponibles.....	33
Conexión a HiveServer2.....	35
Exportar Dashboards como PDF.....	35
Instalación y Comandos.....	35
Requisitos Previos.....	35
Setup Completo (primera vez o reset).....	35
Pasos que ejecuta 'make setup'.....	36
Comandos Disponibles.....	36
Variables de Entorno.....	38
Estructura del repositorio.....	39

Descripción General del Proyecto

Restaurants Analytics es un stack de analítica OLAP que extiende el sistema transaccional del Proyecto 1 (Restaurants-e2). El objetivo es agregar capacidades de análisis de datos distribuido, procesamiento con Apache Spark, orquestación de pipelines con Airflow, análisis de grafos con Neo4J y dashboards interactivos con Metabase.

El sistema consume datos del Proyecto 1 (PostgreSQL, MongoDB y ElasticSearch), los transforma en un Data Warehouse con esquema estrella en Apache Hive, y produce análisis multidimensionales, reportes de tendencias, recomendaciones de productos y rutas optimizadas de entrega.

Componentes Principales

- **Apache Hive:** Data Warehouse con esquema estrella (ORC/SNAPPY)
- **Apache Spark:** Procesamiento distribuido y transformaciones ETL
- **Apache Airflow:** Orquestación del pipeline ETL diario
- **Neo4J:** Base de datos de grafos para recomendaciones y rutas
- **Metabase:** Dashboards interactivos de visualización
- **Docker Compose:** Infraestructura de 8 contenedores

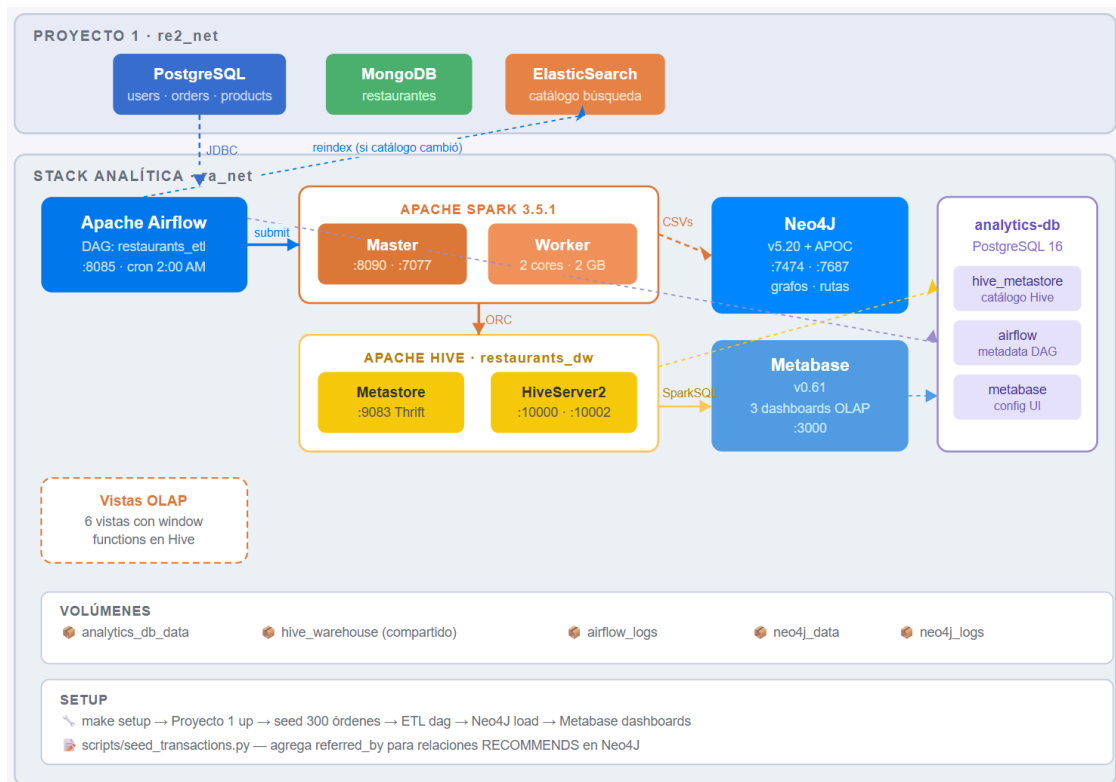
Tecnologías del Proyecto 1 (fuentes de datos)

- **PostgreSQL:** Datos transaccionales: usuarios, órdenes, productos, reservaciones
- **MongoDB:** Datos no relacionales de restaurantes
- **ElasticSearch:** Índice de búsqueda del catálogo de productos

Arquitectura del Sistema

El sistema está dividido en dos stacks Docker interconectados mediante una red externa. El Proyecto 1 expone sus bases de datos a través de la red re2_net, y el stack de analítica se conecta a ella para leer los datos fuente.

Diagrama de Arquitectura



Flujo de Datos	
1	Airflow dispara el DAG restaurants_etl diariamente a las 2:00 AM.
2	Spark lee los datos de Postgres del Proyecto 1 via JDBC.
3	Spark transforma y carga las dimensiones y tablas de hechos en Hive (ORC/SNAPPY).
4	Spark genera CSVs de grafos (usuarios, órdenes, productos, rutas) para Neo4J.
5	Neo4J carga los CSVs y construye el grafo con relaciones PLACED, CONTAINS, BOUGHT_TOGETHER, RECOMMENDS y ROUTE_TO.
6	Si el catálogo de productos cambió, Airflow reindexará ElasticSearch en el Proyecto 1
7	Metabase se conecta a HiveServer2 y sirve dashboards en tiempo real.

Infraestructura Docker

Contenedores

El stack de analítica usa 8 contenedores definidos en deployments/docker-compose.yml:

Contenedor	Imagen	Puerto(s)	Función
ra_analytics_db	postgres:16-alpine	-	Metastore de Hive, metadata de Airflow y Metabase
ra_hive_metastore	Dockerfile.hive	9083	Catálogo Hive vía Thrift
ra_hive_server	Dockerfile.hive	10000, 10002	HiveServer2 / Beeline / UI
ra_spark_master	apache/spark:3.5.1 -python3	8090, 7077	Coordinador de jobs Spark
ra_spark_worker	apache/spark:3.5.1 -python3	-	Ejecutor de transformaciones Spark
ra_airflow	Dockerfile.airflow	8085	Orquestación del pipeline ETL
ra_neo4j	neo4j:5.20	7474, 7687	Base de datos de grafos
ra_metabase	metabase/metabase :v0.61.x	3000	Dashboards de visualización

Redes Docker

ra_net

Red bridge interna. Todos los contenedores del stack de analítica están conectados a esta red. No es accesible desde el Proyecto 1.

re2_net

Red externa del Proyecto 1. Los contenedores que necesitan leer datos del P1 (spark-master, spark-worker, airflow, metabase) están conectados a ambas redes. El nombre exacto de re2_net se detecta automáticamente con make setup y se guarda en .env como RE2_NETWORK_NAME.

Volúmenes Persistentes

Volumen	Propósito
analytics_db_data	Datos de PostgreSQL (hive_metastore, airflow, metabase)
hive_warehouse	Archivos ORC del Data Warehouse. Montado en hive-server, spark-master, spark-worker y airflow para acceso compartido.
airflow_logs	Logs de ejecución del DAG
neo4j_data	Datos del grafo Neo4J
neo4j_logs	Logs de Neo4J

Cadena de Dependencias de Arranque

- analytics-db (healthy)
- hive-metastore (healthy)
 - hive-server
- airflow
- metabase

Dockerfiles Personalizados

Dockerfile.hive

Extiende la imagen oficial de Apache Hive y agrega el driver JDBC de PostgreSQL para que el metastore pueda comunicarse con analytics-db en lugar del Derby embebido.

Dockerfile.airflow

Extiende apache/airflow con providers de Spark, Postgres y MongoDB. También pre-descarga el JAR de PostgreSQL JDBC (/opt/airflow/jars/postgresql-42.7.3.jar) durante el build para evitar descargas en tiempo de ejecución.

Interfaces de Acceso

Servicio	URL	Credenciales
Airflow	http://localhost:8085	admin / admin
Spark Master UI	http://localhost:8090	-
HiveServer2 UI	http://localhost:10002	-
Neo4J Browser	http://localhost:7474	neo4j / Analytics2024!
Metabase	http://localhost:3000	admin@restaurants.local / Admin1234!

Data Warehouse en Apache Hive

El Data Warehouse está implementado en Apache Hive con un esquema estrella. La base de datos se llama `restaurants_dw`. Todos los archivos se almacenan en formato ORC con compresión SNAPPY para maximizar el rendimiento en consultas analíticas.

Esquema Estrella



Dimensiones

dim_tiempo

Generada por Spark a partir de los timestamps de orders y reservations. Permite análisis temporal por año, mes, semana, día y hora.

Columna	Tipo	Descripción
tiempo_key	BIGINT	Llave surrogate (CRC32 del timestamp)
fecha	DATE	Fecha completa
anio / trimestre / mes	INT	Componentes de fecha
nombre_mes	STRING	Ej: Enero, Febrero
semana / día / dia_semana	INT	Semana del año, día del mes, día de la semana (1=Lunes)
nombre_dia	STRING	Ej: Lunes, Martes
hora	INT	Hora del día (0–23)
es_fin_semana	BOOLEAN	TRUE si sábado o domingo

dim_usuario

Columna	Tipo	Descripción
usuario_key	BIGINT	Llave surrogate (CRC32 del UUID)
usuario_id	STRING	UUID original del Proyecto 1
nombre	STRING	Nombre del usuario
rol	STRING	client admin
fecha_registro	DATE	Fecha de creación de la cuenta

dim_restaurante

Columna	Tipo	Descripción
restaurante_key	BIGINT	Llave surrogate
restaurante_id	STRING	UUID original del Proyecto 1
nombre	STRING	Nombre del restaurante
direccion	STRING	Dirección / zona
capacidad	INT	Capacidad máxima de personas

dim_producto

Columna	Tipo	Descripción
producto_key	BIGINT	Llave surrogate
producto_id	STRING	UUID original del Proyecto 1
nombre	STRING	Nombre del producto
categoria	STRING	Categoría del menú
precio_actual	DECIMAL(10,2)	Precio vigente al momento de la carga
disponible	BOOLEAN	Si el producto está activo en el menú
restaurante_id	STRING	Desnormalizado para facilitar joins

Tablas de Hechos

fact_items_pedido

Grano: una fila por ítem dentro de un pedido. Es la tabla de hechos principal y permite análisis de ingresos, tendencias de productos, horarios pico y comparativas por categoría.

Columna	Tipo	Descripción
tiempo_key	BIGINT	FK → dim_tiempo
usuario_key	BIGINT	FK → dim_usuario
restaurante_key	BIGINT	FK → dim_restaurante
producto_key	BIGINT	FK → dim_producto
pedido_id / item_id	STRING	UUIDs originales (trazabilidad)
cantidad	INT	Unidades pedidas
precio_unitario	DECIMAL(10,2)	Precio al momento del pedido (snapshot)
monto_total	DECIMAL(10,2)	cantidad * precio_unitario
estado_pedido	STRING	pending confirmed cancelled (atributo degenerado)
es_para_llevar	BOOLEAN	TRUE si es pickup (atributo degenerado)

fact_reservaciones

Grano: una fila por reservación. Permite análisis de ocupación, tasas de cancelación y comportamiento de clientes.

Columna	Tipo	Descripción
tiempo_key	BIGINT	FK → dim_tiempo (fecha de la reserva)
usuario_key	BIGINT	FK → dim_usuario
restaurante_key	BIGINT	FK → dim_restaurante
reservacion_id	STRING	UUID original (trazabilidad)
tamano_grupo	INT	party_size
estado	STRING	pending confirmed cancelled

Estrategia de Llaves Surrogate (CRC32)

Todas las dimensiones usan llaves surrogate de tipo BIGINT generadas con la función CRC32 de Spark. Esto permite joins eficientes sin UUID strings de 36 caracteres y garantiza reproducibilidad (el mismo UUID siempre produce la misma llave).

```
# Código Spark para generar surrogate key:
from pyspark.sql import functions as F
df = df.withColumn(
    "usuario_key",
    F.crc32(F.col("id").cast("string")).cast("bigint")
)
```

Vistas OLAP

Las vistas OLAP están definidas en `hive/schema/03_olap_views.hql` y se crean sobre las tablas del Data Warehouse. Sirven tanto para los dashboards de Metabase como para consultas ad-hoc con Beeline o SparkSQL.

Vista 1: `olap_ingresos_por_mes_categoria`

Muestra ingresos totales, unidades vendidas, total de pedidos y ticket promedio agrupados por mes y categoría de producto.

```
SELECT
    t.anio, t.mes, t.nombre_mes, p.categoria,
    COUNT(DISTINCT f.pedido_id) AS total_pedidos,
    SUM(f.cantidad) AS unidades_vendidas,
    ROUND(SUM(f.monto_total), 2) AS ingresos_totales,
    ROUND(AVG(f.monto_total), 2) AS ticket_promedio
FROM fact_items_pedido f
JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
JOIN dim_producto p ON f.producto_key = p.producto_key
WHERE f.estado_pedido = 'confirmed'
GROUP BY t.anio, t.mes, t.nombre_mes, p.categoria;
```

Vista 2: `olap_actividad_usuarios_por_restaurante`

Muestra usuarios únicos, total de pedidos e ingresos por restaurante y mes. El restaurante actúa como proxy de zona geográfica.

```
SELECT
    r.nombre AS restaurante, r.direccion AS zona,
    t.anio, t.mes, t.nombre_mes,
    COUNT(DISTINCT f.usuario_key) AS usuarios_unicos,
    COUNT(DISTINCT f.pedido_id) AS total_pedidos,
    ROUND(SUM(f.monto_total), 2) AS ingresos
FROM fact_items_pedido f
JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
JOIN dim_restaurante r ON f.restaurante_key =
    r.restaurante_key
WHERE f.estado_pedido = 'confirmed'
GROUP BY r.nombre, r.direccion, t.anio, t.mes, t.nombre_mes;
```

Vista 3: olap_estado_pedidos

Distribución de pedidos por estado (confirmed, pending, cancelled) con porcentaje calculado mediante función de ventana.

```
SELECT
    t.anio, t.mes, t.nombre_mes, f.estado_pedido,
    COUNT(DISTINCT f.pedido_id) AS total_pedidos,
    ROUND(SUM(f.monto_total), 2) AS monto_total,
    ROUND(
        COUNT(DISTINCT f.pedido_id) * 100.0
        / SUM(COUNT(DISTINCT f.pedido_id))
        OVER (PARTITION BY t.anio, t.mes),
        2
    ) AS porcentaje
FROM fact_items_pedido f
JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
GROUP BY t.anio, t.mes, t.nombre_mes, f.estado_pedido;
```

La función de ventana **SUM(...)** **OVER** (**PARTITION BY** t.anio, t.mes) calcula el total de pedidos del mes como denominador del porcentaje, sin necesidad de un subquery adicional.

Vista 4: olap_tendencias_consumo

Ranking de productos por cantidad vendida dentro de su categoría y mes, usando **RANK()** **OVER** con partición por categoría y mes.

```
SELECT
    t.anio, t.mes, t.nombre_mes,
    p.categoria, p.nombre AS producto,
    SUM(f.cantidad) AS unidades_vendidas,
    ROUND(SUM(f.monto_total), 2) AS ingresos,
    RANK() OVER (
        PARTITION BY t.anio, t.mes, p.categoria
        ORDER BY SUM(f.cantidad) DESC
    ) AS ranking_en_categoria
FROM fact_items_pedido f
JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
JOIN dim_producto p ON f.producto_key = p.producto_key
WHERE f.estado_pedido = 'confirmed'
GROUP BY t.anio, t.mes, t.nombre_mes, p.categoria, p.nombre;
```

Vista 5: olap_horarios_pico

Analiza la distribución de pedidos por hora del día y día de la semana, distinguiendo días de semana de fin de semana.

```
SELECT
    t.hora, t.nombre_dia, t.es_fin_semana,
    COUNT(DISTINCT f.pedido_id) AS total_pedidos,
    SUM(f.cantidad) AS unidades_vendidas,
    ROUND(SUM(f.monto_total), 2) AS ingresos,
    ROUND(AVG(f.monto_total), 2) AS ticket_promedio
FROM fact_items_pedido f
JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
WHERE f.estado_pedido = 'confirmed'
GROUP BY t.hora, t.nombre_dia, t.es_fin_semana;
```

Vista 6: olap_crecimiento_mensual

Calcula el crecimiento mes a mes (MoM) de ingresos usando la función de ventana LAG() para comparar cada mes con el anterior.

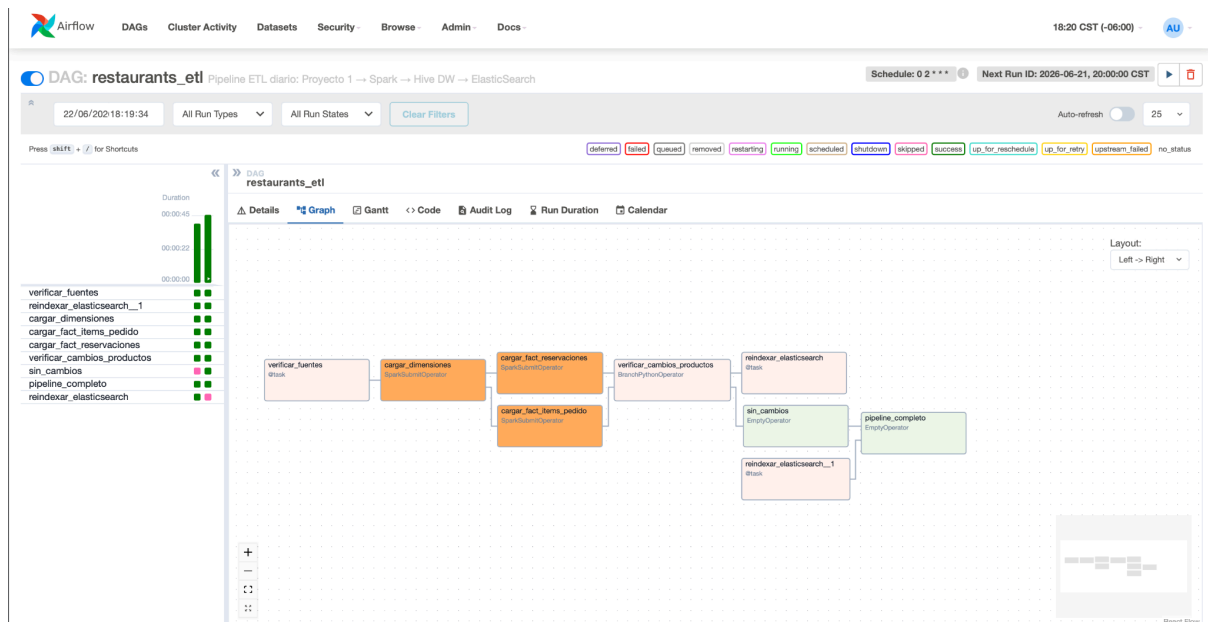
```
SELECT
    anio, mes, nombre_mes,
    ingresos_mes, total_pedidos,
    LAG(ingresos_mes) OVER (ORDER BY anio, mes) AS
    ingresos_mes_anterior,
    ROUND(
        (ingresos_mes - LAG(ingresos_mes) OVER (ORDER BY
        anio, mes))
        / NULLIF(LAG(ingresos_mes) OVER (ORDER BY anio,
        mes), 0) * 100,
        2
    ) AS crecimiento_pct
FROM (
    SELECT t.anio, t.mes, t.nombre_mes,
        ROUND(SUM(f.monto_total), 2) AS ingresos_mes,
        COUNT(DISTINCT f.pedido_id) AS total_pedidos
    FROM fact_items_pedido f
    JOIN dim_tiempo t ON f.tiempo_key = t.tiempo_key
    WHERE f.estado_pedido = 'confirmed'
    GROUP BY t.anio, t.mes, t.nombre_mes
) base;
```

NULLIF(..., 0) evita división por cero en el primer mes o meses sin datos, retornando NULL en lugar de error.

Pipeline ETL con Apache Airflow

El DAG `restaurants_etl` orquesta el pipeline completo de extracción, transformación y carga. Corre automáticamente todos los días a las 2:00 AM (schedule: '0 2 * * *') y puede dispararse manualmente desde la UI de Airflow en <http://localhost:8085>.

Flujo de Tareas



Descripción de cada Tarea

verificar_fuentes

Tarea Python (@task decorator). Obtiene conexión a Postgres del Proyecto 1 vía PostgresHook y ejecuta SELECT COUNT(*) FROM orders. Si falla, el DAG se detiene limpiamente sin ejecutar nada más. Esto previene cargas parciales si la fuente no está disponible.

cargar_dimensiones (SparkSubmitOperator)

Ejecuta spark/jobs/cargar_dimensiones.py. Lee usuarios, restaurantes y productos de Postgres vía JDBC y escribe las cuatro dimensiones en Hive: dim_usuario, dim_restaurante, dim_producto y dim_tiempo. Esta tarea debe completarse antes de las fact tables porque estas hacen JOIN con las dimensiones.

cargar_fact_items_pedido (SparkSubmitOperator)

Ejecuta spark/jobs/cargar_fact_items_pedido.py. Lee orders y order_items de Postgres, hace JOIN con las dimensiones cargadas y escribe fact_items_pedido en Hive. Corre en paralelo con cargar_fact_reservaciones.

cargar_fact_reservaciones (SparkSubmitOperator)

Ejecuta spark/jobs/cargar_fact_reservaciones.py. Lee reservations de Postgres, hace JOIN con dimensiones y escribe fact_reservaciones en Hive. Corre en paralelo con cargar_fact_items_pedido.

verificar_cambios_productos (BranchPythonOperator)

Calcula un hash MD5 del catálogo de productos (id, name, category, price, available ordenados por id) y lo compara con el hash guardado en la Variable de Airflow products_catalog_hash de la ejecución anterior. Si cambia, deriva a reindexar_elasticsearch; si no, a sin_cambios.

reindexar_elasticsearch

Llama al endpoint POST /search/reindex del Proyecto 1 via HTTP. Requiere que el token de admin del API esté guardado en la Variable de Airflow `api_admin_token`. Solo se ejecuta si el catálogo de productos cambia.

pipeline_completo

Tarea vacía (EmptyOperator) con `trigger_rule=NONE_FAILED_MIN_ONE_SUCCESS`. Corre independientemente de cual rama del BranchPythonOperator se ejecutó (reindexar o sin_cambios).

Configuración Compartida de Spark (SPARK_CONF)

```
SPARK_CONF = {
    "spark.sql.catalogImplementation": "hive",
    "spark.sql.warehouse.dir": "/opt/hive/data/warehouse",
    "spark.hadoop.hive.metastore.uris": "thrift://hive-metastore:9083",
    "spark.jars": "/opt/airflow/jars/postgresql-42.7.3.jar",
    "spark.driver.host": "ra-airflow", # (RFC 2396)
    "spark.driver.bindAddress": "0.0.0.0",
    "spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version": "2",
}
```

`spark.driver.host` usa el hostname `ra-airflow` (con guion, no underscore) porque las URIs Java (RFC 2396) rechazan underscores en hostnames y causan Invalid Spark URL en el executor.

El algoritmo de commit v2 (FileOutputCommitter v2) permite que cada executor haga commit de su partición directamente al destino final, sin que el driver tenga que renombrar archivos staging, mejora el rendimiento y la confiabilidad.

Conexiones Requeridas en Airflow

ID	Tipo	Host	Puerto	Schema
postgres_proyecto1	Postgres	re2_postgres	5432	restaurants
spark_default	Spark	spark-master	7077	-

Ambas conexiones se crean automáticamente al ejecutar `make setup`.

Procesamiento con Apache Spark

Apache Spark se utiliza como el motor principal de procesamiento distribuido del proyecto. Su función es transformar los datos transaccionales de órdenes, productos, usuarios y reservaciones en resultados analíticos y en archivos preparados para el grafo de Neo4J. La implementación permite ejecutar el proceso con datos reales desde PostgreSQL/MongoDB o con datos de muestra para validar el flujo sin depender del Proyecto 1.

Archivos relacionados

Archivo	Función dentro del proyecto
spark/jobs/restaurants_spark_analytics.py	Job principal de PySpark. Ejecuta análisis, genera resultados CSV, calcula co-compras y prepara archivos para Neo4J y rutas.
spark/jobs/cargar_dimensiones.py	Carga dimensiones del Data Warehouse en Hive a partir de las fuentes transaccionales.
spark/jobs/cargar_fact_items_pedido.py	Carga la tabla de hechos de ítems de pedidos en Hive.
spark/jobs/cargar_fact_reservaciones.py	Carga la tabla de hechos de reservaciones en Hive.
spark/conf/spark-defaults.conf	Configuración general de Spark, paquetes y ruta de Ivy para dependencias.
spark/conf/hive-site.xml	Configuración de conexión de Spark con el metastore de Hive.
airflow/dags/restaurants_etl.py	DAG que orquesta la ejecución de los jobs Spark dentro del pipeline ETL.

Flujo de procesamiento

Lectura de datos desde la fuente configurada: sample, postgres o mongo.

Normalización de columnas para tener estructuras consistentes entre fuentes.

Construcción de una tabla analítica de ventas llamada fact_sales, uniendo órdenes, productos, usuarios y detalle de pedidos.

Registro de vistas temporales de Spark para ejecutar consultas SparkSQL.

Generación de análisis agregados: tendencias de consumo, horarios pico y crecimiento mensual.

Generación de relaciones de co-compra entre productos a partir de órdenes compartidas.

Simulación de rutas de entrega mediante coordenadas y asignación de pedidos a repartidores.

Exportación de resultados analíticos y archivos CSV que luego consume Neo4J.

Uso de DataFrames y SparkSQL

La implementación combina operaciones de DataFrames con consultas SparkSQL. Los DataFrames se usan para normalizar estructuras, calcular columnas derivadas y preparar información de rutas. SparkSQL se usa para expresar los análisis agregados de forma declarativa, especialmente en ventas, horarios y crecimiento mensual.

Mecanismo	Uso en el proyecto
DataFrames	Normalización de fuentes, joins, cálculo de line_total, ventanas, co-compras y rutas.
SparkSQL	Consultas agregadas sobre fact_sales y reservations para generar resultados analíticos.
Window functions	Cálculo de crecimiento mensual usando LAG sobre ingresos del mes anterior.
Haversine	Cálculo de distancia geográfica entre el centro de distribución y ubicaciones de entrega.

Resultados analíticos generados

Resultado	Descripción	Requisito que cubre
consumption_trends	Agrupar ventas por fecha, categoría y producto. Incluye unidades vendidas, ingresos y cantidad de órdenes.	Tendencias de consumo.
peak_hours	Agrupar órdenes por hora del día para detectar horarios con mayor demanda.	Horarios pico.
monthly_growth	Calcular ingresos por mes, ingreso anterior y porcentaje de crecimiento mensual.	Crecimiento mensual.
reservations_summary	Resumen de reservas por mes y estado, incluyendo cantidad de personas.	Análisis de reservas.
fact_sales	Tabla analítica base con órdenes, productos, usuarios, cantidades e ingresos.	Base para análisis OLAP y exportación.
co_purchases	Calcular pares de productos comprados dentro de una misma orden.	Base para co-compras en Neo4J.
route_assignments	Lista ordenada de paradas por repartidor con distancia y tiempo estimado.	Asignación de rutas de entrega.

Parámetros del job restaurants_spark_analytics.py

Parámetro	Tipo	Requerido	Descripción / Restricción
--source	string	No	Valores permitidos: sample, postgres, mongo. Define de dónde se leen los datos.
--output-base	string	No	Directorio donde Spark escribe los resultados analíticos.
--neo4j-output	string	No	Directorio donde se escriben los CSVs que Neo4J importa.
--couriers	integer	No	Cantidad de repartidores simulados. Debe ser mayor o igual a 1.
--postgres-url	string	Solo postgres	URL JDBC para conectarse a PostgreSQL del Proyecto 1.
--postgres-user	string	Solo postgres	Usuario de PostgreSQL.
--postgres-password	string	Solo postgres	Contraseña de PostgreSQL.
--mongo-uri	string	Solo mongo	URI de conexión a MongoDB.
--center-lat / --center-lon	decimal	No	Coordenadas del centro de distribución para calcular rutas.

--no-hive	flag	No	Omite escritura en Hive y deja solo resultados CSV. Útil para pruebas aisladas.
-----------	------	----	---

Comando de validación en Windows CMD

Para validar el job sin depender de las fuentes reales se puede ejecutar en modo sample. Este comando genera resultados analíticos y también los CSVs necesarios para Neo4J:

```
docker exec -i ra_spark_master /opt/spark/bin/spark-submit
--master spark://spark-master:7077 --conf
spark.driver.host=spark-master
/opt/spark-apps/jobs/restaurants_spark_analytics.py --source
sample --output-base /tmp/restaurants-output --neo4j-output
/opt/neo4j-import --coursiers 2 --no-hive
```

Resultado esperado: el job debe finalizar con el mensaje [OK] Job Spark finalizado correctamente y debe generar carpetas bajo /tmp/restaurants-output/results, además de CSVs bajo /opt/neo4j-import.

```
[OK] Job Spark finalizado correctamente
[OK] Resultados analíticos generados en: /tmp/restaurants-output/results
[OK] CSVs para Neo4J generados en: /opt/neo4j-import

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_spark_master bash -lc "find /tmp/restaurants-output/results -maxdepth 2 -type d | sort"
/tmp/restaurants-output/results
/tmp/restaurants-output/results/consumption_trends
/tmp/restaurants-output/results/co_purchases
/tmp/restaurants-output/results/fact_sales
/tmp/restaurants-output/results/monthly_growth
/tmp/restaurants-output/results/peak_hours
/tmp/restaurants-output/results/reservations_summary
/tmp/restaurants-output/results/route_assignments

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_spark_master bash -lc "find /opt/neo4j-import -maxdepth 1 -type f -name '*.csv' | sort"
/opt/neo4j-import/co_purchases.csv
/opt/neo4j-import/locations.csv
/opt/neo4j-import/order_items.csv
/opt/neo4j-import/orders.csv
/opt/neo4j-import/products.csv
/opt/neo4j-import/recommendations.csv
/opt/neo4j-import/route_assignments.csv
/opt/neo4j-import/route_edges.csv
/opt/neo4j-import/users.csv

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>
```


Análisis de Grafos con Neo4J

Neo4J se utiliza para representar relaciones que son más naturales como grafo que como tablas: usuarios que realizan pedidos, pedidos que contienen productos, productos comprados juntos, recomendaciones entre usuarios y ubicaciones conectadas por rutas de entrega. Esto permite ejecutar consultas Cypher para identificar patrones de co-compra, usuarios influyentes y caminos entre ubicaciones.

Archivos relacionados

Archivo	Función
neo4j/queries/00_constraints.cypher	Define restricciones de unicidad para evitar duplicados en nodos principales.
neo4j/queries/01_load_graph.cypher	Carga nodos y relaciones desde los CSVs generados por Spark.
neo4j/queries/02_analysis_queries.cypher	Consultas de análisis: co-compras, recomendadores, usuarios activos, categorías y caminos de entrega.
neo4j/queries/03_route_queries.cypher	Consultas específicas para rutas: ruta ordenada, resumen por repartidor y tramos entre ubicaciones.
neo4j/queries/04_visualization_queries.cypher	Consultas diseñadas para visualizar grafos en Neo4J Browser.
neo4j/import/*.csv	Archivos generados por Spark. Son entrada para Neo4J y no deben editarse manualmente.

Estructura del grafo

Tipo de nodo	Descripción
User	Usuario o cliente que realiza pedidos y puede recomendar a otros usuarios.
Product	Producto disponible en el catálogo.
Order	Pedido realizado por un usuario.

Courier	Repartidor simulado para la asignación de entregas.	
Location	Geonodo que representa el centro de distribución o una ubicación de entrega.	
RouteStop	Parada específica dentro de la ruta de un repartidor.	
Relación	Origen → Destino	Descripción
PLACED	User → Order	Indica que un usuario realizó un pedido.
CONTAINS	Order → Product	Indica los productos incluidos en un pedido. Guarda cantidad, precio unitario y total.
RECOMMENDS	User → User	Representa una recomendación o referencia entre usuarios.
BOUGHT_TOGETHER	Product → Product	Conecta productos que fueron comprados juntos en la misma orden.
DELIVERS_TO	Order → Location	Conecta una orden con su ubicación de entrega.
HAS_STOP	Courier → RouteStop	Asocia un repartidor con las paradas asignadas.
DELIVERS_ORDER	RouteStop → Order	Relaciona cada parada con el pedido entregado.
ROUTE_TO	Location → Location	Representa un tramo de ruta con distancia y tiempo estimado.

Carga del grafo

La carga de Neo4J se divide en dos etapas: primero se aplican constraints para garantizar unicidad, y luego se cargan los nodos y relaciones desde los CSVs generados por Spark.

```
docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\00_constraints.cypher

docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\01_load_graph.cypher
```

Resultado esperado: Neo4J debe crear nodos User, Product, Order, Courier, Location y RouteStop, junto con sus relaciones principales.

```
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\00_constraints.cypher
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>echo === CARGANDO GRAFO ===
=== CARGANDO GRAFO ===

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\01_load_graph.cypher
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>echo === VALIDANDO NODOS ===
=== VALIDANDO NODOS ===

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! "MATCH (n) RETURN labels(n) AS tipo, count(n) AS cantidad ORDER BY cantidad DESC;"
echo === VALIDANDO RELACIONES ===
docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! "MATCH ()-[r]->() RETURN type(r) AS relacion, count(r) AS cantidad ORDER BY cantidad DESC;"
tipo, cantidad
["Order"], 8
["Location"], 8
["RouteStop"], 7
["Product"], 6
["User"], 5
["Courier"], 2
```

Consultas de análisis implementadas

Consulta	Archivo	Propósito
Productos más comprados juntos	02_analysis_queries.cyphe r	Obtiene los cinco pares de productos con mayor frecuencia de co-compra.
Usuarios que recomiendan a otros	02_analysis_queries.cyphe r	Identifica usuarios que generan recomendaciones o referencias.
Usuarios con mayor actividad	02_analysis_queries.cyphe r	Cuenta pedidos realizados por usuario.
Categorías más compradas	02_analysis_queries.cyphe r	Calcula unidades e ingresos por categoría desde el grafo.

Caminos desde el centro de distribución	02_analysis_queries.cyphe r	Consulta rutas desde el geonodo central hacia ubicaciones de entrega.
Ruta ordenada por repartidor	03_route_queries.cypher	Muestra cada parada en orden por repartidor.
Resumen de rutas por repartidor	03_route_queries.cypher	Resume pedidos asignados, distancia total y tiempo total.
Tramos entre geonodos	03_route_queries.cypher	Muestra cada tramo de ruta entre ubicaciones.

Comandos de validación en Windows CMD

```
docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\02_analysis_queries.cypher
```

```
docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\03_route_queries.cypher
```

Resultado esperado: las consultas deben devolver tablas con productos comprados juntos, usuarios recomendadores, categorías, caminos de entrega, rutas por repartidor y resumen de distancias/tiempos.

```
C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>echo === CONSULTAS DE ANALISIS ===
=== CONSULTAS DE ANALISIS ===

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < neo4j\queries\02_analysis_queries.cypher
producto_1, producto_2, veces_comprados_juntos
"Hamburguesa cl?sica", "Papas medianas", 2
"Pizza margarita", "Brownie", 2
"Pizza pepperoni", "Limonada natural", 2
"Hamburguesa cl?sica", "Brownie", 1
"Hamburguesa cl?sica", "Limonada natural", 1
usuario_id, usuario, usuarios_recomendados, recomendados
"u1", "Ana", 2, ["Luis", "Sof?a"]
"u2", "Luis", 1, ["Marco"]
usuario_id, usuario, pedidos_realizados
"u1", "Ana", 2
"u2", "Luis", 2
"u3", "Sof?a", 2
"u4", "Marco", 1
"u5", "Valeria", 1
categoria, unidades, ingresos
"pizzas", 6, 28400.0
"hamburguesas", 4, 15600.0
"bebidas", 6, 10800.0
"postres", 4, 6400.0
"acompanamientos", 4, 6000.0
destino, ruta, distancia_total_km, tiempo_total_min
"Entrega o6", ["Centro de distribuci?n", "Entrega o1", "Entrega o6"], 3.221, 7.73
"Entrega o1", ["Centro de distribuci?n", "Entrega o1"], 3.221, 7.73
"Entrega o2", ["Centro de distribuci?n", "Entrega o2"], 5.287, 12.69
"Entrega o7", ["Centro de distribuci?n", "Entrega o2", "Entrega o7"], 5.287, 12.69
"Entrega o3", ["Centro de distribuci?n", "Entrega o1", "Entrega o6", "Entrega o3"], 13.185, 31.64
```

```

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>echo === CONSULTAS DE RUTAS ===
=== CONSULTAS DE RUTAS ===

C:\Users\beatr\Documents\Github\SharedRepos\Restaurants-analytics>docker exec -i ra_neo4j cypher-shell -u neo4j -p Analytics2024! < n
eo4j\queries\03_route_queries.cypher
repartidor, parada, pedido, ubicacion, zona, distancia_desde_anterior_km, tiempo_estimado_min, distancia_acumulada_km
"courier-1", 1, "o1", "Entrega o1", "San Pedro", 3.221, 7.73, 3.221
"courier-1", 2, "o6", "Entrega o6", "San Pedro", 0.0, 0.0, 3.221
"courier-1", 3, "o3", "Entrega o3", "Escaz?", 9.964, 23.91, 13.184
"courier-1", 4, "o8", "Entrega o8", "Escaz?", 0.0, 0.0, 13.184
"courier-2", 1, "o2", "Entrega o2", "Curridabat", 5.287, 12.69, 5.287
"courier-2", 2, "o7", "Entrega o7", "Curridabat", 0.0, 0.0, 5.287
"courier-2", 3, "o5", "Entrega o5", "Cartago", 13.832, 33.2, 19.119
repartidor, pedidos_asignados, distancia_total_km, tiempo_total_min
"courier-1", 4, 13.185, 31.64
"courier-2", 3, 19.119, 45.89
repartidor, parada, desde, hacia, distancia_km, tiempo_min
"courier-1", 1, "Centro de distribuci?n", "Entrega o1", 3.221, 7.73
"courier-1", 2, "Entrega o1", "Entrega o6", 0.0, 0.0
"courier-1", 3, "Entrega o6", "Entrega o3", 9.964, 23.91
"courier-1", 4, "Entrega o3", "Entrega o8", 0.0, 0.0
"courier-2", 1, "Centro de distribuci?n", "Entrega o2", 5.287, 12.69
"courier-2", 2, "Entrega o2", "Entrega o7", 0.0, 0.0
"courier-2", 3, "Entrega o7", "Entrega o5", 13.832, 33.2

```

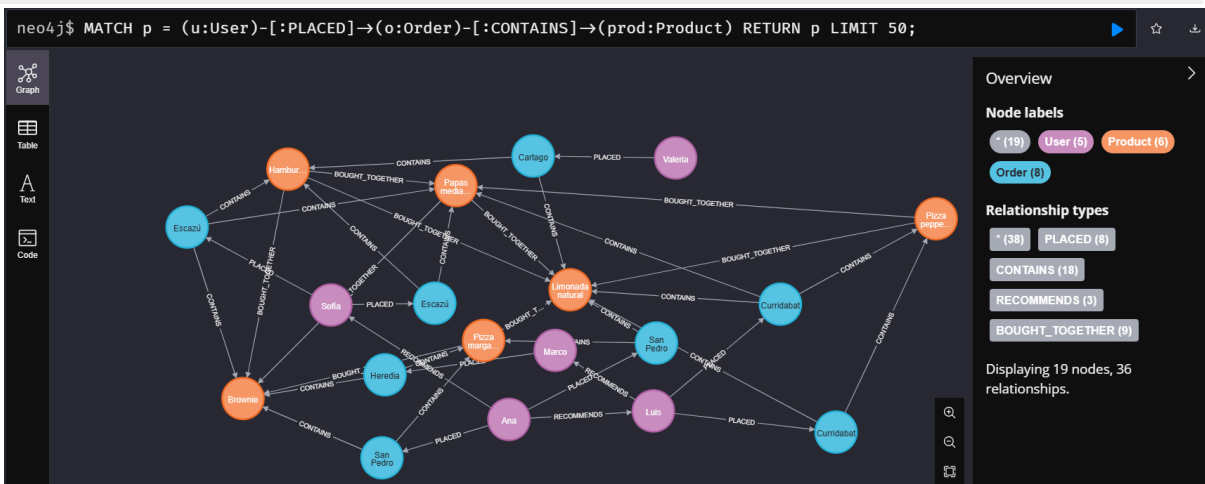
Consultas visuales en Neo4J Browser

Para generar evidencias visuales, se abre Neo4J Browser en <http://localhost:7474> con el usuario neo4j y contraseña Analytics2024!. Las consultas visuales se ejecutan en modo Graph.

```

MATCH
    (u:User) -[:PLACED]->(o:Order) -[:CONTAINS]->(prod:Product)
RETURN p
LIMIT 50;

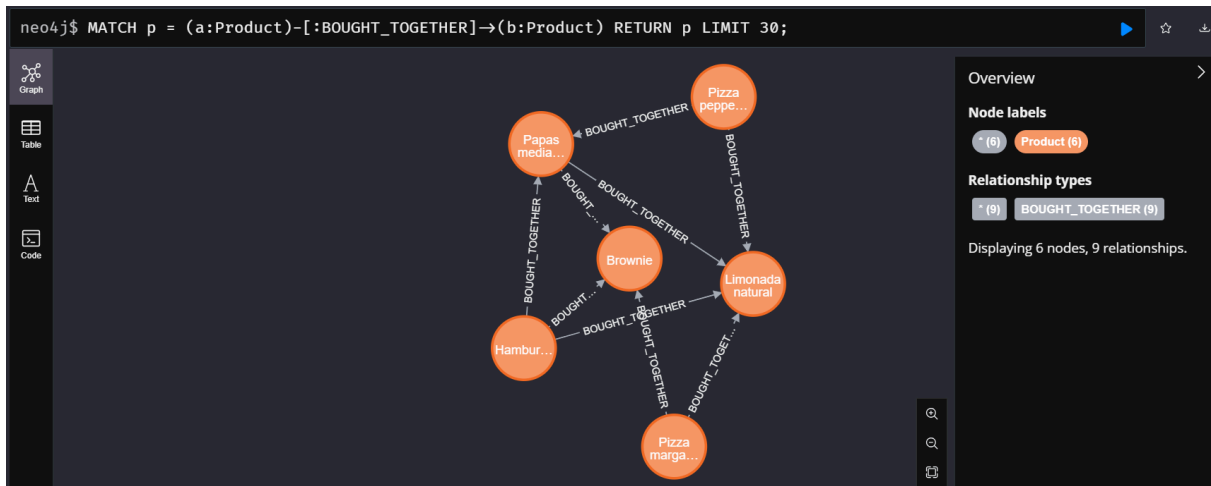
```



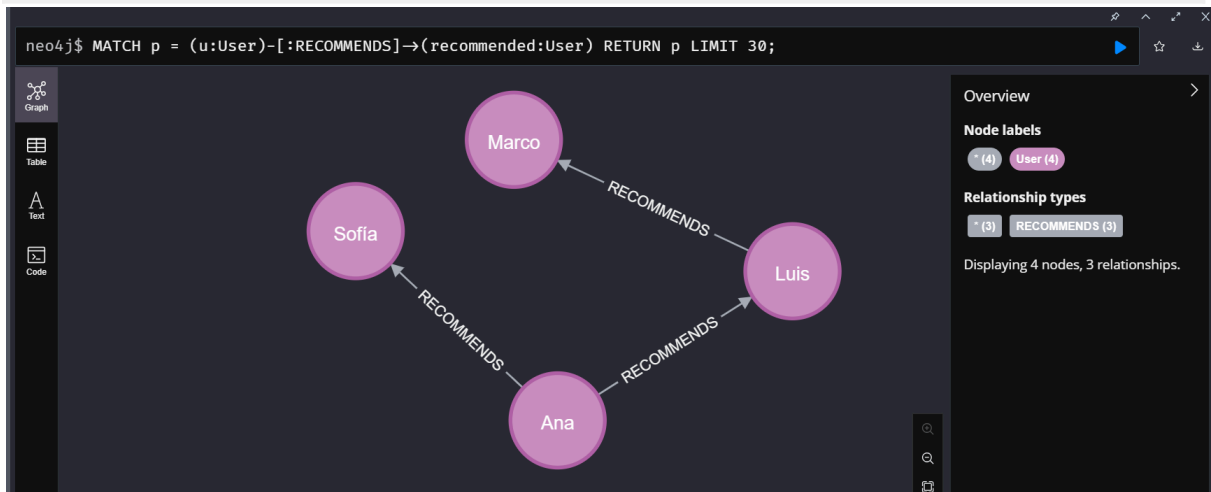
```

MATCH p = (a:Product) -[:BOUGHT_TOGETHER]->(b:Product)
RETURN p
LIMIT 30;

```



```
MATCH p = (u:User)-[:RECOMMENDS]->(recommended:User)
RETURN p
LIMIT 30;
```



Asignación de Rutas de Entrega

La asignación de rutas de entrega se implementó dentro del job de Spark y luego se modeló en Neo4J para poder consultarla y visualizarla como grafo. El objetivo es simular rutas para pedidos con geolocalización de clientes y asignarlas a repartidores de forma coherente.

Heurística utilizada

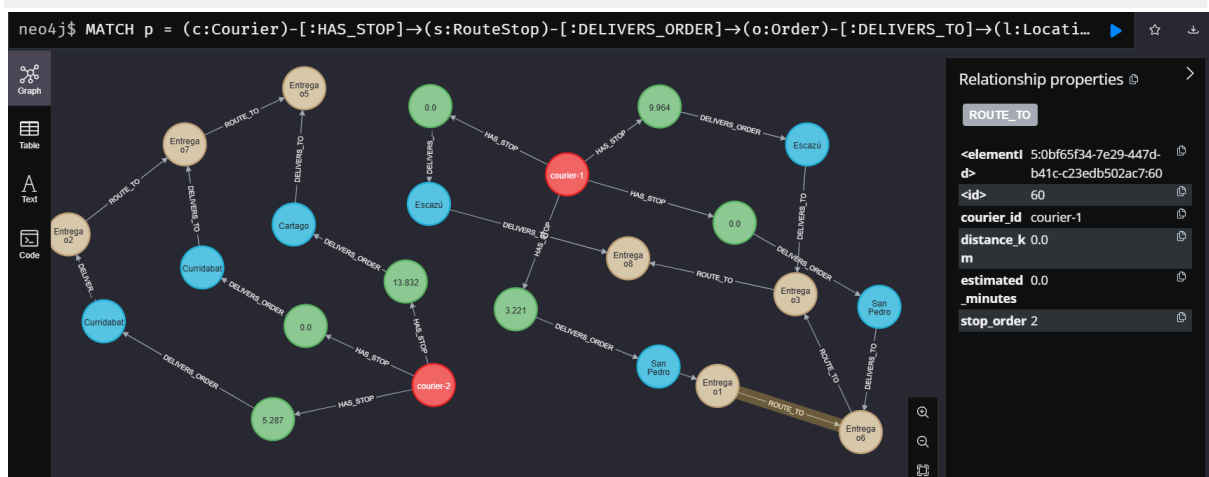
La solución usa una heurística de vecino más cercano. Para cada repartidor, se parte desde un centro de distribución y se selecciona como siguiente parada la ubicación pendiente más cercana. Después de cada selección, la ubicación actual se actualiza y el proceso se repite hasta terminar las entregas asignadas.

La distancia entre coordenadas se calcula con la fórmula de Haversine, adecuada para estimar distancias entre puntos geográficos usando latitud y longitud. El tiempo estimado se calcula a partir de una velocidad promedio definida dentro del job.

Archivo generado	Contenido
route_assignments.csv	Ruta ordenada por repartidor: parada, pedido, usuario, ubicación, zona, distancia desde la parada anterior, tiempo estimado y distancia acumulada.
route_edges.csv	Tramos entre ubicaciones: origen, destino, repartidor, número de parada, distancia y tiempo estimado.

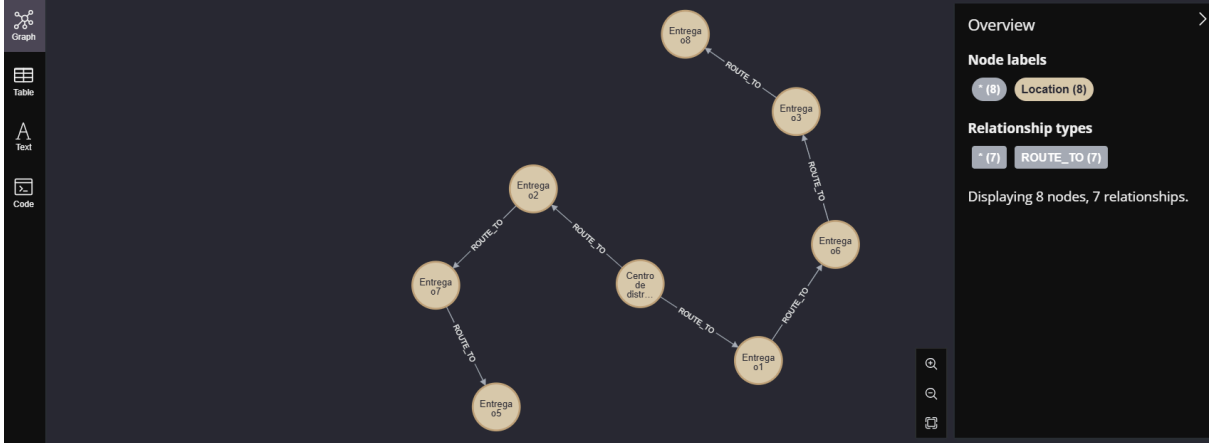
Validación visual de rutas

```
MATCH
    p
    (c:Courier)-[:HAS_STOP]->(s:RouteStop)-[:DELIVERS_ORDER]->(o:Order)-[:DELIVERS_TO]
    ]->(l:Location)
    RETURN p
    LIMIT 60;
```



```
MATCH p = (:Location)-[:ROUTE_TO]->(:Location)
RETURN p
LIMIT 60;
```

```
neo4j$ MATCH p = (:Location)-[:ROUTE_TO]->(:Location) RETURN p LIMIT 60;
```



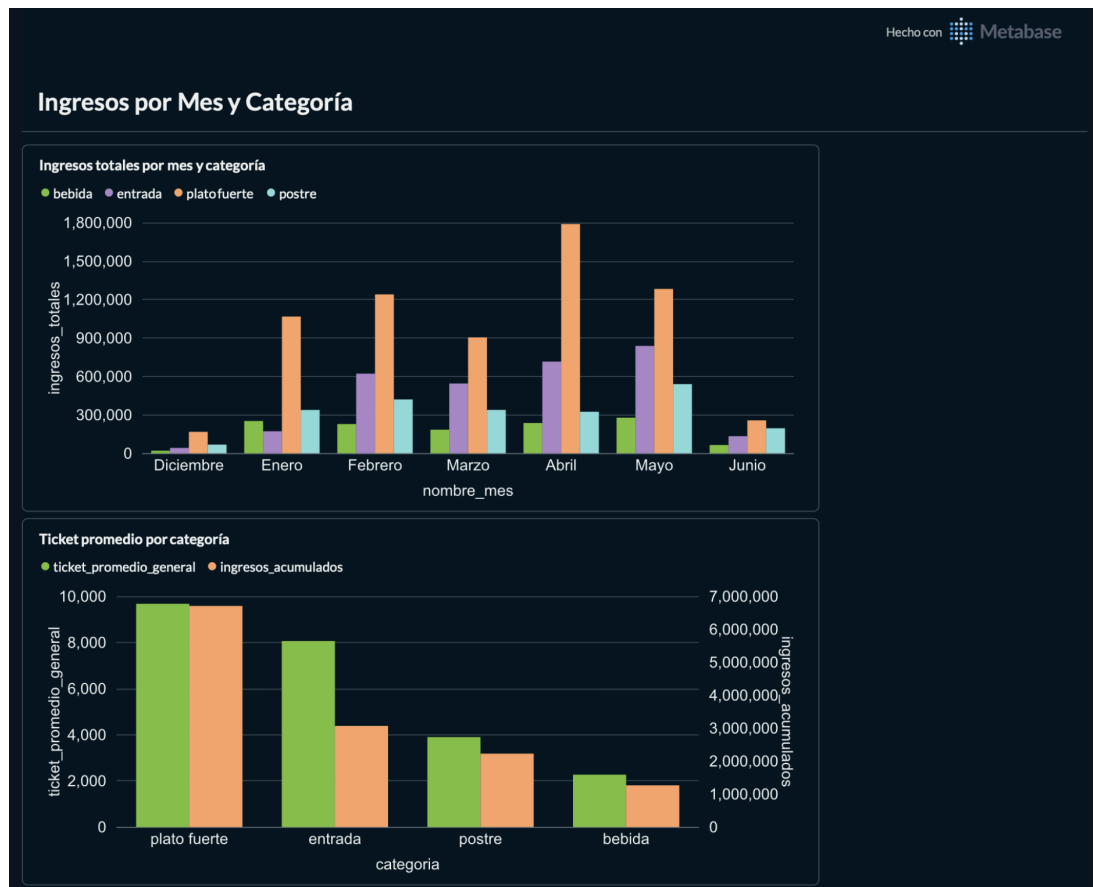
Visualización con Metabase

Metabase v0.61.x se conecta a HiveServer2 vía SparkSQL (puerto 10000) y expone tres dashboards con las vistas OLAP del Data Warehouse. Los dashboards y conexiones se configuran automáticamente al final de make setup mediante `dashboards/metabase/setup_metabase.py`.

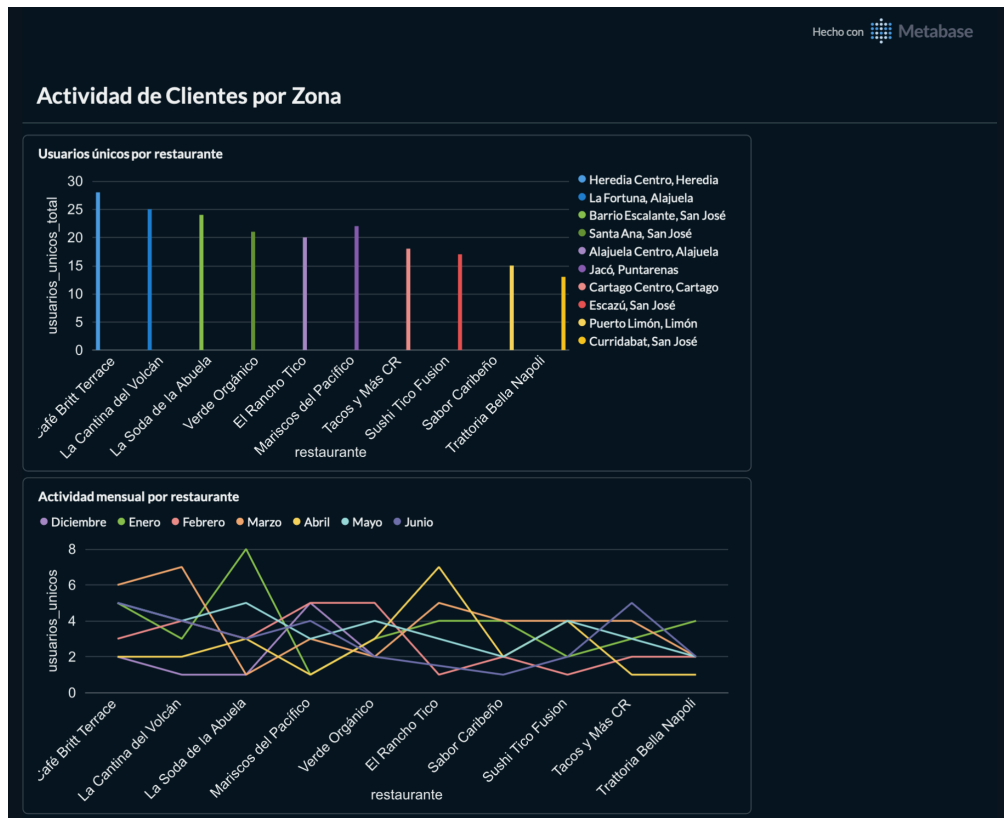
Dashboards Disponibles

Dashboard	Contenido
Ingresos por Mes y Categoría	Ingresos totales por mes y categoría, productos mas vendidos, ticket promedio, ventas por hora (olap_ingresos_por_mes_categoria + olap_tendencias_consumo)
Actividad de Clientes por Zona	Usuarios únicos y pedidos por restaurante/zona, mapa de calor de actividad mensual (olap_actividad_usuarios_por_restaurante)
Estado de Pedidos	Distribución confirmed/pending/cancelled con porcentajes, tendencia de crecimiento mensual (olap_estado_pedidos + olap_crecimiento_mensual)

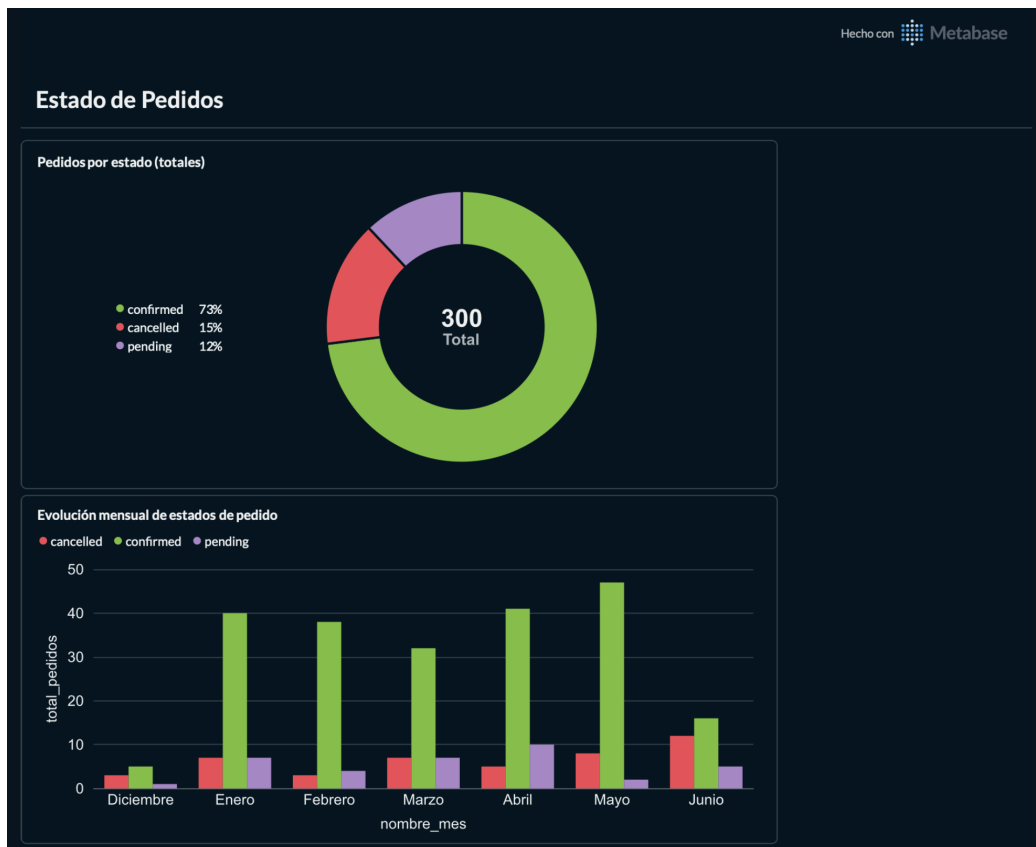
Ingresos por Mes y Categoría



Actividad de Clientes por Zona



Estado de Pedidos



Conexión a HiveServer2

Campo	Valor
Tipo de base de datos	SparkSQL
Host	hive-server
Puerto	10000
Base de datos	restaurants_dw

Exportar Dashboards como PDF

Desde cualquier dashboard en Metabase: hacer click en el menu de opciones (tres puntos) → Exportar como PDF. Esto genera un archivo PDF con todos los graficos del dashboard en su estado actual.

Instalación y Comandos

Requisitos Previos

- Docker 24+ y Docker Compose v2
- Make (macOS y Linux; en Windows usar WSL2)
- Python 3 con cryptography y psycopg2-binary
- Repositorio del Proyecto 1 (Restaurants-e2) levantado
- Mínimo 8 GB de RAM disponibles para Docker

Setup Completo (primera vez o reset)

```
git clone https://github.com/c4dd3/Restaurants-analytics
cd Restaurants-analytics
make setup
```

El proceso tarda entre 5 y 25 minutos. Al finalizar imprime las URLs y credenciales de acceso.

Si el Proyecto 1 esta en una ruta diferente a ../Restaurants-e2:

```
make setup P1=/ruta/al/proyecto1
```

Pasos que ejecuta 'make setup'

Paso	Acción
1	Baja el stack de analítica con volúmenes (limpieza total)
2	Baja el Proyecto 1 con volúmenes
3	Levanta el Proyecto 1 y espera que su API responda
4	Siembra datos base: 10 restaurantes, 2 menus c/u, 8 productos c/u, 20 usuarios
5	Siembra 300 órdenes y 150 reservaciones aleatorias
6	Configura .env automáticamente (red re2, llaves de Airflow) y levanta el stack de analítica
7	Crea el esquema Hive: dimensiones, tablas de hechos y vistas OLAP
8	Despansa y ejecuta el DAG en Airflow; espera hasta que termine (max. 20 min)
9	Carga el grafo en Neo4J y configura conexiones y dashboards de Metabase

Comandos Disponibles

Comando	Descripción
make setup	Levanta todo desde cero (primera vez o reset)
make demo-reset	Alias exacto de make setup
make teardown	Baja Proyecto 1 y Proyecto 2 con volúmenes
make up	Levanta solo el stack de analítica
make down	Baja el stack conservando volúmenes
make down-v	Baja el stack y elimina volúmenes
make logs	Sigue los logs de todos los servicios
make ps	Estado de todos los contenedores
make beeline	Abre Beeline conectado a HiveServer2

<code>make neo4j-shell</code>	Abre Cypher Shell en Neo4J
<code>make spark-shell</code>	Abre Spark Shell en el master
<code>make neo4j-load</code>	Recarga constraints y grafo desde los CSVs
<code>make neo4j-analysis</code>	Ejecuta las queries de análisis de grafos
<code>make spark-job-sample</code>	Corre el job de Spark con datos de muestra (sin Postgres)

Variables de Entorno

Variable	Descripcion	Default
RE2_NETWORK_NAME	Red Docker del Proyecto 1	auto-detectado
AIRFLOW_FERNET_KEY	Llave para cifrar conexiones	auto-generada
AIRFLOW_SECRET_KEY	Llave para sesiones web	auto-generada
ANALYTICS_DB_USER	Usuario de analytics-db	analytics
ANALYTICS_DB_PASSWORD	Contraseña de analytics-db	analytics
NEO4J_PASSWORD	Contraseña de Neo4J	Analytics2024!
SPARK_WORKER_MEMORY	Memoria por worker	2G
SPARK_WORKER_CORES	Cores por worker	2

Estructura del repositorio

```
Restaurants-analytics/
├── airflow/
│   ├── dags/
│   │   └── restaurants_etl.py      # Pipeline ETL orquestado
│   ├── plugins/
│   │   ├── extractors/           # Extracción desde Postgres y MongoDB
│   │   └── loaders/              # Carga a Hive y Elasticsearch
│   └── requirements.txt
├── dashboards/
│   └── metabase/
│       └── setup_metabase.py      # Configura conexiones y dashboards via API
├── deployments/
│   ├── docker-compose.yml
│   ├── Dockerfile.airflow        # Airflow + Java + JDBC driver pre-descargado
│   ├── Dockerfile.hive
│   ├── airflow/
│   │   └── entrypoint.sh          # Inicializa DB y arranca webserver + scheduler
│   └── postgres/
│       └── init-multiple-dbs.sh   # Crea hive_metastore, airflow y metabase
├── docs/                          # Documentación técnica y diagramas
├── hive/
│   └── schema/
│       ├── 01_dimensions.hql     # DDL de dimensiones
│       ├── 02_facts.hql          # DDL de tablas de hechos
│       └── 03_olap_views.hql     # Vistas OLAP precalculadas
├── neo4j/
│   ├── import/                   # CSVs generados por Spark para carga del grafo
│   └── queries/
│       ├── 00_constraints.cypher  # Índices y constraints
│       ├── 01_load_graph.cypher   # Carga inicial del grafo
│       └── 02_analysis_queries.cypher
├── scripts/
│   ├── configure_env.py          # Auto-genera llaves de Airflow y detecta red re2
│   └── seed_transactions.py       # Siembra órdenes y reservaciones en Proyecto 1
├── spark/
│   ├── conf/
│   │   ├── hive-site.xml         # Apunta Spark al metastore de Hive
│   │   └── spark-defaults.conf
│   ├── jobs/
│   │   ├── cargar_dimensiones.py
│   │   ├── cargar_fact_items_pedido.py
│   │   ├── cargar_fact_reservaciones.py
│   │   └── restaurants_spark_analytics.py # Genera CSVs para Neo4J
│   └── utils/
├── .env.example
├── Makefile
└── README.md
```

Repositorio

<https://github.com/c4dd3/Restaurants-analytics>